

Error Amplification Limits ANN-to-SNN Conversion in Continuous Control

Zijie Xu^{1,2} Zihan Huang² Yiting Dong² Kang Chen² Wenxuan Liu² Zhaofei Yu^{1,2}

Abstract

Spiking Neural Networks (SNNs) can achieve competitive performance by converting already existing well-trained Artificial Neural Networks (ANNs), avoiding further costly training. This property is particularly attractive in Reinforcement Learning (RL), where training through environment interaction is expensive and potentially unsafe. However, existing conversion methods perform poorly in continuous control, where suitable baselines are largely absent. We identify error amplification as the key cause: small action approximation errors become temporally correlated across decision steps, inducing cumulative state distribution shift and severe performance degradation. To address this issue, we propose Cross-Step Residual Potential Initialization (CRPI), a lightweight gradient-free mechanism that carries over residual membrane potentials across decision steps to suppress temporally correlated errors. Experiments on continuous control benchmarks with both vector and visual observations demonstrate that CRPI can be integrated into existing conversion pipelines and substantially recovers lost performance. Our results highlight continuous control as a critical and challenging benchmark for ANN-to-SNN conversion, where small errors can be strongly amplified and impact performance. Code is available at <https://github.com/xuzijie32/ANN2SNN-CRPI>.

1. Introduction

Spiking Neural Networks (SNNs) (Maass, 1997; Gerstner et al., 2014) communicate through discrete spikes rather than continuous activations, enabling event-driven computa-

¹Institute for Artificial Intelligence, Peking University, Beijing, China ²Beijing Key Laboratory of Brain-inspired Spiking Large Models, School of Computer Science, Peking University, Beijing, China. Correspondence to: Wenxuan Liu, Zhaofei Yu <liuwx66@pku.edu.cn, yuzf12@pku.edu.cn>.

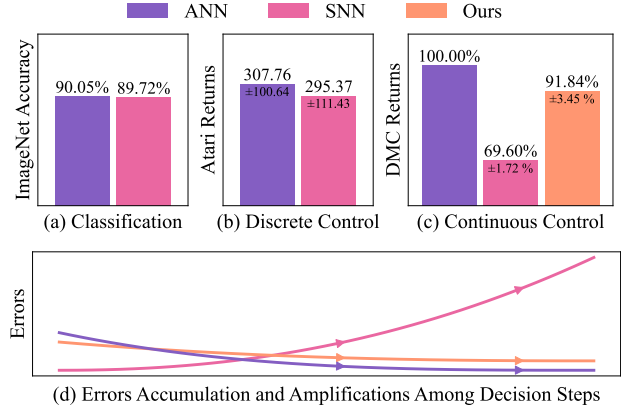


Figure 1. Challenges of ANN-to-SNN conversion across different task categories. (a) Classification accuracy on ImageNet (Huang et al., 2025). (b) Average returns in discrete control tasks on Atari (Patel et al., 2019). (c) Relative returns in continuous control tasks, averaged over six environments from the DeepMind Control Suite. Additional results in the experimental section confirm that the performance degradation is consistent across tasks. (d) Illustration of error accumulation and amplification, where trajectories generated by converted SNNs progressively diverge from those of the original ANN policies.

tion and substantially reducing energy consumption when deployed on neuromorphic hardware (Merolla et al., 2014; Davies et al., 2018; DeBole et al., 2019). These properties make SNNs particularly attractive for Reinforcement Learning (RL) on resource-constrained edge devices such as drones, wearables, and Internet-of-Things (IoT) sensors, where power efficiency is critical (Xu et al., 2026a;b; Song et al., 2026; Li et al., 2026).

ANN-to-SNN conversion constructs SNNs by transferring pretrained ANN weights and replacing nonlinear activations with spiking neurons, allowing SNNs to inherit strong ANN performance without additional training (Cao et al., 2015; Han et al., 2020; Li et al., 2021; Deng & Gu, 2021; Bu et al., 2022a; 2025). This gradient-free paradigm is especially valuable in RL, where learning an agent typically requires extensive environment interaction that is costly, time-consuming, and potentially unsafe (Jiang et al., 2023; Padalkar et al., 2024; Tang et al., 2025; Jayant & Bhatnagar, 2022). By reusing pretrained ANN policies, ANN-to-SNN conversion allows energy-efficient SNN agents to be deployed without extensive environment interaction.

Despite these advantages, the study of ANN-to-SNN conversion in RL remains limited in scope. Existing work has primarily focused on discrete control settings (Patel et al., 2019; Tan et al., 2021; Kumar et al., 2025; Feng et al., 2024), while ANN-to-SNN conversion in continuous action spaces remains largely unexplored, despite its central role in real-world robotics and embodied AI systems (Kober et al., 2013; Gu et al., 2017; Brunke et al., 2022). Figure 1(a)–(c) compares the performance of ANN-to-SNN conversion across classification, discrete control, and continuous control tasks. While existing conversion methods achieve competitive performance in classification and discrete control, they suffer substantially larger performance degradation in continuous control. This gap arises from the requirement for precise, high-dimensional vector-valued actions in continuous control, in contrast to the categorical outputs in classification and discrete control tasks, making continuous control considerably more sensitive to conversion errors.

To understand this phenomenon, we conduct a detailed analysis of conversion errors in continuous control. We find that: (i) performance degradation in converted SNNs is primarily driven by deviations in induced state trajectories rather than instantaneous action errors; (ii) these state deviations grow progressively over decision steps along a trajectory; and (iii) action approximation errors exhibit positive temporal correlation across consecutive decision steps, amplifying even small conversion errors. As illustrated in Figure 1(d), trajectories generated by converted SNNs gradually diverge from those of the optimal ANN policies, whereas ANN policies themselves do not exhibit such progressive drift.

Motivated by this analysis, we propose Cross-Step Residual Potential Initialization (CRPI), a simple yet effective mechanism to mitigate error amplification in ANN-to-SNN conversion for RL. CRPI carries over residual membrane potentials across consecutive decision steps to initialize neuron states, suppressing temporally correlated action errors and stabilizing the resulting state trajectories, as illustrated in Figure 1(d). Notably, CRPI requires no additional training and can be seamlessly integrated into existing gradient-free ANN-to-SNN conversion pipelines.

We evaluate CRPI on a range of continuous control benchmarks, including vector-based tasks from MuJoCo (Todorov et al., 2012) and vision-based environments from the DeepMind Control (DMC) Suite (Tunyasuvunakool et al., 2020). CRPI consistently improves the performance of multiple state-of-the-art ANN-to-SNN conversion methods and outperforms directly trained SNNs in challenging vision-based continuous control tasks. Our results highlight continuous control as a challenging benchmark for ANN-to-SNN conversion, where conversion errors can be strongly amplified and significantly impact long-horizon performance.

2. Related Works

2.1. ANN–SNN Conversion

ANN-to-SNN conversion typically maps ReLU activations in ANNs to the firing rates of Integrate-and-Fire neurons by accumulating spikes over time (Cao et al., 2015). However, the bounded firing rates in SNNs introduce significant errors, which is often mitigated through techniques such as weight normalization (Rueckauer et al., 2017) and threshold balancing (Han et al., 2020). Temporal discretization introduces additional quantization errors, which have been addressed by methods like quantizing the source ANN activations (Bu et al., 2023; Hu et al., 2023), using two-stage inference (Hao et al., 2023a), improving membrane potential initialization (Hao et al., 2023b), and extending neuron models with signed spikes (Wang et al., 2022a; Li et al., 2022) or multiple thresholds (Huang et al., 2024). Other encoding schemes such as time-to-first-spike coding (Rueckauer & Liu, 2018; Zhang et al., 2019; Stanojevic et al., 2023), phase coding (Kim et al., 2018; Wang et al., 2022b), burst coding (Park et al., 2019; Li & Zeng, 2022; Wang et al., 2025), and differential coding (Huang et al., 2025), have also been explored to enhance both efficiency and expressiveness. Recent works have further extended conversion methods by allowing for approximations of general nonlinear layers (Oh & Lee, 2024; Jiang et al., 2024; Huang et al., 2024) and enabling conversion of Transformer architectures to SNNs (Wang et al., 2023; You et al., 2024).

2.2. SNNs for Reinforcement Learning

Early works on SNNs for RL primarily relied on biologically inspired local learning rules, particularly reward-modulated spike-timing-dependent plasticity (R-STDP) and its variants (Florian, 2007; Frémaux & Gerstner, 2016; Gerstner et al., 2018; Frémaux et al., 2013; Yang et al., 2024). Later research introduced gradient-based optimization methods, such as spatio-temporal backpropagation (STBP) for deep spiking Q-networks (Wu et al., 2018; Liu et al., 2022; Chen et al., 2022; Qin et al., 2022; Sun et al., 2022) and e-prop for policy gradient methods (Bellec et al., 2020). Qin et al. 2025 further introduces gated recurrent mechanisms and demonstrates strong performance on partially observable tasks. In continuous control, the hybrid actor-critic framework has been widely adopted, where a spiking actor network is co-trained with an ANN-based critic network (Xu et al., 2026a;b; Tang et al., 2020, 2021; Chen et al., 2024; Ding et al., 2022). This approach has been further advanced with proxy-target mechanisms, allowing spiking actors to match or even surpass ANN policies (Xu et al., 2026a). Additionally, recent studies have explored proper normalization to stabilize and improve SNNs in both discrete and continuous control (Xu et al., 2026b).

2.3. ANN-SNN Conversion in Reinforcement Learning

ANN-to-SNN conversion in RL has also been explored in several studies. These works mainly focus on converting Deep Q-Networks (DQNs) (Mnih, 2013; Mnih et al., 2015) into spiking policies for Atari games (Patel et al., 2019; Tan et al., 2021), as well as deploying converted agents in real-world robotic tasks, such as ball catching (Feng et al., 2024) and path planning (Kumar et al., 2025). These studies report competitive performance, improved energy efficiency, and enhanced robustness of SNN-based agents. However, existing works have been limited to discrete control tasks, and ANN-to-SNN conversion in continuous control remains largely unexplored. This work shows that directly applying existing conversion techniques to continuous control leads to greater performance degradation, which forms the primary motivation for our work.

3. Preliminaries

3.1. Spiking Neural Networks

SNNs process information via discrete spike events and temporal membrane dynamics. For an Integrate-and-Fire (IF) neuron in layer l at discrete time step t , the neuronal dynamics are given by

$$\mathbf{I}^l[t] = \mathbf{W}^l \mathbf{x}^{l-1}[t] + \mathbf{b}^l, \quad (1)$$

$$\mathbf{m}^l[t] = \mathbf{v}^l[t-1] + \mathbf{I}^l[t], \quad (2)$$

$$\mathbf{o}^l[t] = H(\mathbf{m}^l[t] - \boldsymbol{\theta}^l), \quad (3)$$

$$\mathbf{x}^l[t] = \boldsymbol{\theta}^l \odot \mathbf{o}^l[t], \quad (4)$$

$$\mathbf{v}^l[t] = \mathbf{m}^l[t] - \mathbf{x}^l[t], \quad (5)$$

where $\mathbf{x}^l[t]$ denotes the post-synaptic potential, $\mathbf{m}^l[t]$ and $\mathbf{v}^l[t]$ are the pre-reset and post-reset membrane potentials respectively, $\mathbf{o}^l[t]$ is the binary spike output, and $\boldsymbol{\theta}^l$ is the firing threshold. The operator $H(\cdot)$ denotes the Heaviside step function, and $\odot$ indicates element-wise multiplication.

3.2. ANN-to-SNN Conversion

ANN-to-SNN conversion leverages the correspondence between ReLU activations in ANNs and averaged firing responses in rate-coded SNNs. In a standard feedforward ANN, the output of layer l is computed as

$$\mathbf{z}^l = \text{ReLU}(\mathbf{W}_{\text{ANN}}^l \mathbf{z}^{l-1} + \mathbf{b}_{\text{ANN}}^l). \quad (6)$$

Starting from the discrete-time dynamics of IF neurons, the membrane potential update can be written as

$$\mathbf{v}^l[t] = \mathbf{v}^l[t-1] + \mathbf{W}^l \mathbf{x}^{l-1}[t] + \mathbf{b}^l - \mathbf{x}^l[t]. \quad (7)$$

Averaging this equation over time steps $t = 1$ to T yields

$$\frac{1}{T} \sum_{t=1}^T \mathbf{x}^l[t] = \mathbf{W}^l \frac{1}{T} \sum_{t=1}^T \mathbf{x}^{l-1}[t] + \mathbf{b}^l + \frac{\mathbf{v}^l[0] - \mathbf{v}^l[T]}{T}. \quad (8)$$

Under the standard assumption that the membrane potential of IF neurons remains bounded by the firing threshold, i.e., $\mathbf{v}^l[t] \in [0, \boldsymbol{\theta}^l)$, the residual term $\frac{\mathbf{v}^l[0] - \mathbf{v}^l[T]}{T}$ vanishes as T increases. By setting $\mathbf{W}^l = \mathbf{W}_{\text{ANN}}^l$ and $\mathbf{b}^l = \mathbf{b}_{\text{ANN}}^l$, and identifying ANN activations with the average post-synaptic potential $\mathbf{z}^{l-1} = \frac{1}{T} \sum_{t=1}^T \mathbf{x}^{l-1}[t]$, the time-averaged SNN response $\frac{1}{T} \sum_{t=1}^T \mathbf{x}^l[t]$ converges to the corresponding ANN activation $\mathbf{z}^l$.

3.3. Reinforcement Learning

RL studies the problem of an agent interacting with an environment, which is commonly formalized as a Markov Decision Process (MDP). At decision step k , the agent observes the environment state $\mathbf{s}_k \in \mathcal{S}$ and selects an action $\mathbf{a}_k \in \mathcal{A}$ according to a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$. The environment then transitions to a new state $\mathbf{s}_{k+1}$ and provides a reward $r_k = r(\mathbf{s}_k, \mathbf{a}_k)$. The objective of the agent is to maximize the expected cumulative return $R = \mathbb{E} \sum_k r_k$.

A key property of the MDP formulation is that the environment dynamics and reward depend only on the current state and action, rather than the full history of past interactions. Accordingly, at each decision step, the policy computes an action solely based on the current observation. In standard ANN-to-SNN conversion for RL, this is typically enforced by executing the SNN for a fixed internal simulation horizon of T time steps at each decision step, and initializing all neuronal states at the beginning of the next decision step. As a result, no internal neuronal states or membrane potentials are preserved across consecutive decision steps.

4. Analyzing the Conversion Errors

This section investigates error propagation in ANN-to-SNN conversion for continuous control and identifies a phenomenon of error amplification. Section 4.1 decomposes the performance degradation of converted SNN policies into instantaneous action errors and the resulting state distribution shift, showing that the latter dominates the return loss. Section 4.2 demonstrates that small approximation errors are amplified across decision steps, leading to great state distribution shift. Section 4.3 identifies positive temporal correlations in action errors across consecutive decisions as the underlying cause of this amplification.

4.1. State-Dominated Performance Degradation

Given that ANN-to-SNN conversion exhibits greater performance degradation in continuous control than in widely-

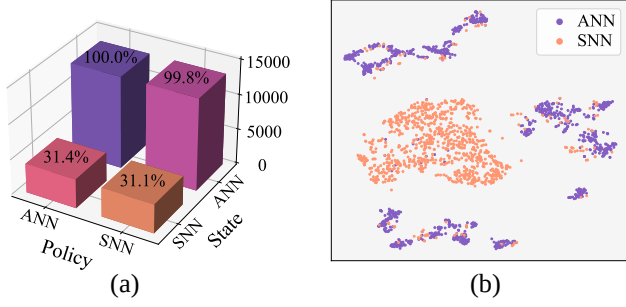


Figure 2. Analysis of performance degradation in ANN-to-SNN conversion in the HalfCheetah-v4 environment. The ANN policy is trained with TD3 for 3 million environment steps and converted using IF neurons with 8 simulation steps. (a) Expected returns under different combinations of policies and state distributions. (b) t-SNE visualization of state trajectories induced by ANN and converted SNN policies, revealing significant distribution divergence.

studied image classification tasks, we pose a central question: **Is this error solely due to the conversion process, or is it also amplified by the dynamics of RL environments?**

We begin by formalizing the expected return of a policy π :

$$R^\pi = \mathbb{E}_{s \sim P_\pi(s), a = \pi(s)} [r(s, a)], \quad (9)$$

where $P_\pi(s)$ denotes the marginal state distribution induced by executing policy π in the environment. Accordingly, the expected returns of the original ANN policy and its converted SNN counterpart are given by

$$R^{\text{ANN}} = \mathbb{E}_{s \sim P_\pi^{\text{ANN}}(s), a = \pi^{\text{ANN}}(s)} [r(s, a)], \quad (10)$$

$$R^{\text{SNN}} = \mathbb{E}_{s \sim P_\pi^{\text{SNN}}(s), a = \pi^{\text{SNN}}(s)} [r(s, a)]. \quad (11)$$

The discrepancy between R^{ANN} and R^{SNN} arises from two sources: (i) divergence in the state visitation distributions, i.e., P_π^{ANN} versus P_π^{SNN} , and (ii) differences in action selection induced by the converted policy, i.e., π^{ANN} versus π^{SNN} . To disentangle these effects, we define two auxiliary returns:

$$R^{\text{SNN}|\text{ANN}} = \mathbb{E}_{s \sim P_\pi^{\text{ANN}}(s), a = \pi^{\text{SNN}}(s)} [r(s, a)], \quad (12)$$

$$R^{\text{ANN}|\text{SNN}} = \mathbb{E}_{s \sim P_\pi^{\text{SNN}}(s), a = \pi^{\text{ANN}}(s)} [r(s, a)]. \quad (13)$$

The **SNN-action-only return** $R^{\text{SNN}|\text{ANN}}$ evaluates the effect of replacing the ANN policy with the converted SNN policy while keeping the ANN-induced state distribution fixed. Conversely, the **SNN-state-only return** $R^{\text{ANN}|\text{SNN}}$ isolates the impact of state distribution shift induced by the converted SNN while preserving the original ANN policy.

Figure 2(a) reports the expected returns of the ANN, SNN, SNN-action-only, and SNN-state-only settings. Replacing π^{ANN} with π^{SNN} while maintaining the ANN-induced state distribution results in only negligible performance degradation (less than 0.5%). In contrast, executing either policy

under the SNN-induced state distribution leads to a substantial reduction in return. Comprehensive experiments results in Appendix B.1 also demonstrates same pattern exists across diverse RL environments and SNN settings. This indicates that the performance degradation is overwhelmingly dominated by deviations in the induced state trajectories rather than instantaneous action mismatches. Furthermore, Fig. 2(b) visualizes the divergence between state trajectories generated by ANN and SNN policies. Despite their close per-step action outputs, the resulting trajectories diverge greatly, highlighting the sensitivity of continuous control systems to small perturbations.

4.2. Error Accumulation and Amplification

Having identified state distribution shift as the primary source of performance degradation, a natural question arises: **how does this shift evolve along a trajectory? Is it uniformly distributed, or does it grow over time?**

In a Markov Decision Process, each action affects the subsequent state, which in turn influences all future transitions. Consequently, small action errors introduced by ANN-to-SNN conversion can propagate across decision steps, causing progressive state deviations and accumulating performance loss.

Figure 3 (a) illustrates how state trajectories induced by ANN and converted SNN policies diverge over time. The discrepancy is small at early stages but grows progressively as interaction proceeds, demonstrating that state deviations accumulate and are amplified by the environments. Figure 3 (b) then shows the corresponding impact on returns. While ANN and SNN policies achieve similar rewards at the start of an episode, the gap steadily widens over the decision horizon, reflecting the cumulative effect of state divergence on long-horizon performance.

4.3. Positive Cross-Step Correlation

The analysis in Section 4.2 shows that conversion errors accumulate and amplify over decision steps. A natural question arises: **why do these errors persist instead of being corrected by subsequent actions?**

We analyze the correlation of action approximation errors across consecutive steps. At step k with state s_k , let the actions produced by the ANN and the converted SNN be a_k^{ANN} and a_k^{SNN} , and define the instantaneous action error as $\delta a_k = a_k^{\text{SNN}} - a_k^{\text{ANN}}$. Executing these actions leads to next states s_{k+1}^{ANN} and s_{k+1}^{SNN} . To separate the effect of policy response from state shift, we define a counterfactual action $a_{k+1}^{\text{cf}} = \pi^{\text{ANN}}(s_{k+1}^{\text{SNN}})$, which applies the ANN policy to the SNN-induced next state.

Using these definitions, we compute three cosine similarity metrics that characterize cross-step error propagation:

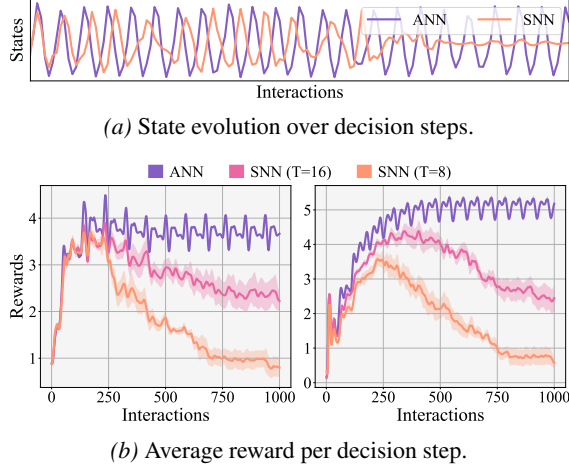


Figure 3. (a) One-dimensional visualization of state evolution over decision steps for ANN and converted SNN policies in the HalfCheetah-v4 environment, obtained by projecting paired trajectories onto the first principal component via PCA. (b) Average reward per decision step for ANN and converted SNN policies in Hopper-v4 (left) and Walker2d-v4 (right). Shaded regions denote half a standard deviation. All curves are uniformly smoothed for clarity. The ANN was trained with TD3 for 3 million environment steps, and the SNN uses IF neurons.

Table 1. Cosine similarity of action errors across consecutive decision steps in different environments. The ANN is trained with TD3 for 3 million environment steps and the SNN uses IF neurons with 16 simulation steps.

ENVIRONMENT	ANN CORRECTION	SNN CONSISTENCY	SNN DRIFT
ANT-V4	-0.188 ± 0.004	0.276 ± 0.018	0.030 ± 0.019
HALFCHEETAH-V4	-0.070 ± 0.023	0.101 ± 0.033	0.043 ± 0.043
HOPPER-V4	-0.256 ± 0.005	0.481 ± 0.016	0.129 ± 0.013
WALKER2D-V4	-0.185 ± 0.009	0.462 ± 0.015	0.180 ± 0.020

ANN Correction measures whether the ANN policy compensates for the previous-step action error under the shifted state:

$$\text{ANN Correction} = \cos(\delta a_k, a_{k+1}^{\text{cf}} - a_{k+1}^{\text{ANN}}). \quad (14)$$

SNN Consistency captures whether the SNN exhibits similar action deviations across consecutive steps under the same shifted state:

$$\text{SNN Consistency} = \cos(\delta a_k, a_{k+1}^{\text{SNN}} - a_{k+1}^{\text{cf}}). \quad (15)$$

SNN Drift directly quantifies the temporal correlation of action errors between ANN and SNN trajectories:

$$\text{SNN Drift} = \cos(\delta a_k, a_{k+1}^{\text{SNN}} - a_{k+1}^{\text{ANN}}). \quad (16)$$

Intuitively, negative ANN Correction indicates that the ANN policy actively compensates for previous errors, whereas positive SNN Consistency shows that errors persist across

steps. Table 1 confirms this: ANN policies display negative temporal correlations, demonstrating inherent error-correcting behavior, while converted SNNs exhibit positive correlations (SNN Consistency) and consistent drift (SNN Drift). These results suggest that temporally correlated action errors are the key mechanism behind error accumulation and amplification in ANN-to-SNN conversion.

5. Reducing the Compounding Errors

The analysis in Section 4 shows that the performance degradation of ANN-to-SNN conversion in continuous control is primarily driven by positively correlated action approximation errors across consecutive decision steps. Once such temporal correlations arise, even small conversion errors are repeatedly reinforced by the environment dynamics, inducing progressive state drift and ultimately leading to amplified performance loss. Our objective is therefore to explicitly suppress this cross-step error correlation.

5.1. Deriving the Methods

Motivated by the prior analyses of rate-based ANN-to-SNN conversion (Bu et al., 2022b), we assume that the dominant source of action approximation error arises from residual membrane potentials at the end of each decision step k in ANN-to-SNN conversion:

$$\varepsilon_k^l = \frac{\mathbf{v}_k^l[T] - \mathbf{v}_k^l[0]}{T}. \quad (17)$$

We use the temporal correlation of residual membrane potential errors as a tractable proxy for action-level error correlation.

Empirical results in Section 4.3 indicate that these residual errors exhibit positive temporal correlation, i.e., $\mathbb{E}[\cos(\varepsilon_{k+1}^l, \varepsilon_k^l)] > 0$, which directly leads to systematic error accumulation across decision steps. Rather than minimizing the magnitude of ε_k^l independently at each step, we instead aim to suppress its temporal correlation. Formally, our goal is to enforce

$$\mathbb{E}[\cos(\tilde{\varepsilon}_{k+1}^l, \varepsilon_k^l)] \leq 0, \quad (18)$$

where $\tilde{\varepsilon}_{k+1}^l$ denotes the modified residual error after applying a cross-step correction.

To this end, we consider a first-order approximation of residual error dynamics across consecutive decision steps:

$$\tilde{\varepsilon}_{k+1}^l = \varepsilon_{k+1}^l - \alpha \varepsilon_k^l, \quad (19)$$

where $\alpha > 0$ captures the empirically observed positive alignment between successive residual errors (as demonstrated in Table 1). Increasing α reduces the expected cosine similarity in Eq. (18) and can drive it below zero, thereby suppressing error accumulation.

Algorithm 1 Inference with CRPI

Initialize membrane potentials for all layers $\mathbf{v}_0^l[0] \leftarrow \frac{1}{2}\theta^l$
 Observe initial environment state s_0
 Run SNN for T steps and execute action $a_0 = \pi^{\text{SNN}}(s_0)$
for $k = 1$ to K **do**
 Observe next state s_k
 Compute residual potential from previous step:

$$\Delta \mathbf{v}_k^l \leftarrow \mathbf{v}_{k-1}^l[T] - \mathbf{v}_{k-1}^l[0]$$

Clip residual to ensure valid firing range:

$$\Delta \mathbf{v}_k^l \leftarrow \max(\Delta \mathbf{v}_k^l, -\sum_{t=1}^T \mathbf{x}_{k-1}^l[t])$$

Initialize membrane potentials with cross-step residual:

$$\mathbf{v}_k^l[0] \leftarrow \text{clip}(\frac{1}{2}\theta^l + \alpha \Delta \mathbf{v}_k^l, 0, \theta^l)$$

Run SNN for T steps and execute $a_k = \pi^{\text{SNN}}(s_k)$
end for

Substituting the definition of ε_k^l into Eq. (19) yields

$$\tilde{\varepsilon}_{k+1}^l = \frac{\mathbf{v}_{k+1}^l[T] - \mathbf{v}_{k+1}^l[0]}{T} - \alpha \frac{\mathbf{v}_k^l[T] - \mathbf{v}_k^l[0]}{T} \quad (20)$$

$$= \frac{\mathbf{v}_{k+1}^l[T] - (\mathbf{v}_{k+1}^l[0] - \alpha (\mathbf{v}_k^l[T] - \mathbf{v}_k^l[0]))}{T}. \quad (21)$$

Under the standard assumption that the final membrane potential is approximately uniformly distributed in $(0, \theta^l)$ (Bu et al., 2022a) and weakly dependent on its initialization (supported by empirical evidence in Appendix B.2), the expected residual error can be reduced by adjusting the initial membrane potential as

$$\mathbf{v}_{k+1}^l[0] \leftarrow \mathbf{v}_{k+1}^l[0] + \alpha (\mathbf{v}_k^l[T] - \mathbf{v}_k^l[0]). \quad (22)$$

5.2. Cross-Step Residual Potential Initialization

We refer to the membrane potential initialization mechanism of Equation (22) as Cross-Step Residual Potential Initialization (CRPI). In practice, Equation (17) assumes non-negative activations induced by ReLU. Therefore, we clip $\mathbf{v}_k^l[T] - \mathbf{v}_0^l[T]$ in Equation (22) to a minimum of $-\sum_{t=1}^T \mathbf{x}_k^l[t]$ to remain consistent with Equation (6). Moreover, to prevent excessively large residuals from inducing abnormally high initial membrane potentials that may cause persistent bursting across subsequent decision steps, we further clip $\mathbf{v}_{k+1}^l[0]$ to the valid membrane potential range. The complete procedure is summarized in Algorithm 1.

CRPI introduces no additional training or architectural modification and operates solely through membrane potential initialization. It is lightweight and can be readily integrated

with existing ANN-to-SNN conversion techniques such as normalization, quantization, and neuron model extensions.

6. Experiments

6.1. Experimental Setup

Environments. We evaluate CRPI on a diverse set of continuous control benchmarks with both vector-based and vision-based observations. For vector-based control, we consider four standard MuJoCo environments (Todorov et al., 2012; Todorov, 2014) from OpenAI Gymnasium (Brockman, 2016; Towers et al., 2024): Ant (Schulman, 2015), HalfCheetah (Wawrzynski, 2009), Hopper (Erez et al., 2012), and Walker2d. For vision-based control, we evaluate six tasks from the DeepMind Control (DMC) Suite (Tunyasuvunakool et al., 2020): Cartpole_Swingup, Finger_Spin, Reacher_Easy, Cheetah_Run, Acrobot_Swingup, and Quadruped_Walk.

RL Algorithms. For vector-based environments, we use ANN policies pre-trained with three sample-efficient off-policy algorithms: Deep Deterministic Policy Gradient (DDPG) (Lillicrap, 2015), Twin Delayed DDPG (TD3) (Fujimoto et al., 2018), and Soft Actor-Critic (SAC) (Haarnoja et al., 2018a;b). Each policy is trained for 3 million environment steps. For vision-based environments, we adopt ANN policies trained with Data-Regularized Q-v2 (DrQ-v2) (Yarats et al., 2021b;a) for 1 million environment steps.

ANN-to-SNN Conversion Methods. We integrate CRPI with multiple ANN-to-SNN conversion techniques. As a baseline, we apply CRPI to standard IF neurons. To assess compatibility with more expressive neuron models, we further combine CRPI with Signed Neuron Models (SNM) (Wang et al., 2022a), Multi-Threshold Neurons (MT) (Huang et al., 2024), and Differential Coding (DC) (Huang et al., 2025). All MT neurons use four firing thresholds. To avoid additional hyperparameter tuning, the firing threshold θ is set to the maximum ReLU activation channel-wise.

Evaluation Protocol. All results are averaged over five random seeds. For each seed, we evaluate the policy over ten rollout episodes of up to 1,000 interaction steps (terminated earlier if the episode ends), yielding a total of 50,000 environment steps per method. The hyperparameter α is selected via a coarse grid search over $\{0, 0.1, 0.2, \dots, 0.9, 1.0\}$ and is fixed across all seeds.

6.2. Reducing Error Correlation

To evaluate the effectiveness of CRPI in mitigating the temporally correlated conversion errors identified in Section 4, we first examine how CRPI influences cross-step error correlation. Specifically, we analyze the cosine similarity of

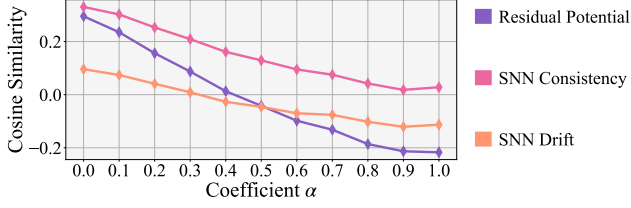


Figure 4. Cosine similarity of residual membrane potential and action errors across consecutive decision steps under different values of α . Results are obtained on MuJoCo environments using TD3 and IF neurons with 16 simulation steps.

residual membrane potential errors across adjacent decision steps, together with the *SNN Consistency* and *SNN Drift* metrics introduced in Section 4.3, which quantify temporal correlation at the action level.

Figure 4 shows that the temporal correlation of residual membrane potential errors decreases monotonically as α increases, indicating that CRPI effectively suppresses cross-step error correlation. Importantly, this reduction at the membrane level consistently propagates to the action level: both the *SNN Consistency* and *SNN Drift* metrics are substantially reduced as α increases. These results provide direct empirical evidence that CRPI decorrelates conversion-induced errors across decision steps, addressing the root cause of error amplification identified in Section 4.3.

6.3. Enhancing Performance

We next examine how this reduction in temporal error correlation translates into policy performance. Figure 5 illustrates the relationship between the correlation coefficient α and relative performance (the converted SNN’s performance normalized by the ANN baseline) across tasks. Starting from $\alpha = 0$ (standard ANN-to-SNN conversion), increasing α leads to a steady improvement in performance, demonstrating that incorporating cross-step residual information effectively mitigates temporally correlated conversion errors. However, when α becomes too large, performance begins to degrade. That is because excessive values of α overcompensate residual errors from previous steps, resulting in unstable membrane potential initialization and greater output error. This behavior reveals a clear trade-off between error decorrelation and overcorrection.

Overall, these results show that CRPI prevents small approximation errors from being repeatedly reinforced by closed-loop system dynamics. By suppressing temporal error correlation, CRPI alleviates the error amplification phenomenon analyzed in Section 4.2 and stabilizes long-horizon behavior in continuous control tasks, which we identified as the dominant factor governing performance degradation in Section 4.1.

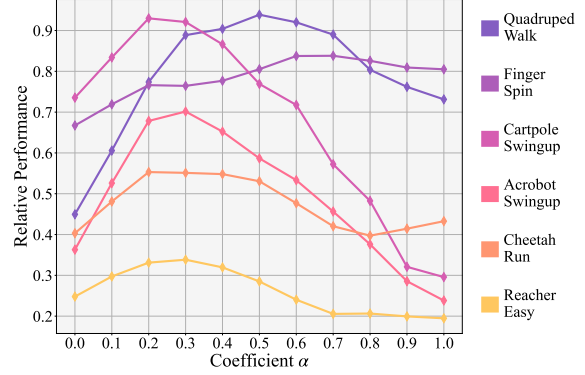


Figure 5. Relative performance on DeepMind Control tasks under different correlation parameter α , using IF neurons with 32 simulation steps. Performance is normalized by the corresponding ANN return. Curves are uniformly smoothed for visualization.

6.4. Comprehensive Benchmarks

We conduct comprehensive evaluations on both MuJoCo and DeepMind Control Suite (DMC) benchmarks to assess the effectiveness of CRPI under a wide range of settings, including different observation modalities, neuron models, SNN simulation steps, environments, and underlying RL algorithms. Tables 2 and 3 report the performance of the converted SNN policies on MuJoCo and DMC tasks. For each configuration, we report the Average Performance Ratio (APR), defined as the mean ratio of SNN performance to the corresponding ANN performance, expressed in percentage and averaged across all evaluated environments.

Across all evaluated configurations, CRPI consistently improves the performance ratio compared to the corresponding baseline ANN-to-SNN conversion methods¹. These improvements are observed consistently across different neuron models, simulation lengths, and RL algorithms, indicating that CRPI is robust to both architectural and algorithmic variations. Detailed results for individual MuJoCo environments are provided in Appendix B.3, where CRPI demonstrates consistent performance gains across all tasks.

Table 3 additionally includes comparisons with state-of-the-art directly trained SNNs on visual-based DMC tasks, including approaches incorporating a world model (Hafner et al., 2019). Specifically, we compare against vanilla Leaky Integrate-and-Fire (LIF) neurons, the two-component spiking neuron model (TC-LIF) (Zhang et al., 2024), and

¹In several cases, the converted SNNs slightly outperform their ANN counterparts. This phenomenon may be attributed to the inherent stochasticity of RL environments, where the event-driven dynamics and robustness of SNNs can better tolerate noise (Ding et al., 2025). Similar observations have also been reported in prior studies on ANN-to-SNN conversion for discrete control tasks (Patel et al., 2019; Tan et al., 2021).

Error Amplification Limits ANN-to-SNN Conversion in Continuous Control

Table 2. Performance comparison of the average performance ratio of ANN-to-SNN conversion on MuJoCo continuous control tasks.

NEURON	T	CONVERTING DDPG			CONVERTING TD3			CONVERTING SAC		
		ORIGINAL	OURS	Δ	ORIGINAL	OURS	Δ	ORIGINAL	OURS	Δ
IF	8	77.96%	87.78%	+9.82%	64.71%	72.26%	+7.55%	70.25%	75.38%	+5.13%
	16	82.57%	95.41%	+12.84%	79.11%	85.10%	+5.99%	88.80%	93.09%	+4.29%
	32	94.24%	105.07%	+10.83%	89.68%	93.52%	+3.84%	97.98%	99.70%	+1.71%
SNM	8	65.69%	76.72%	+11.04%	82.97%	89.24%	+6.27%	79.56%	87.89%	+8.34%
	16	95.14%	102.06%	+6.92%	96.70%	98.18%	+1.48%	92.13%	98.09%	+5.96%
MT	2	68.74%	86.74%	+18.00%	76.29%	80.18%	+3.89%	84.02%	90.06%	+6.03%
	4	85.68%	102.22%	+16.54%	97.93%	98.09%	+0.16%	95.29%	97.44%	+2.15%
DC	2	74.32%	83.97%	+9.65%	89.39%	93.65%	+4.27%	86.58%	92.93%	+6.35%
	4	101.65%	104.81%	+3.16%	99.30%	99.80%	+0.50%	96.85%	101.72%	+4.88%

Table 3. Performance comparison on DeepMind Control Suite tasks with visual observations, where $\pm$ denotes half a standard deviation.

MODULE	T	CRPI	ACROBOT SWINGUP	CARTPOLE SWINGUP	CHEETAH RUN	FINGER SPIN	QUADRUPED WALK	REACHER EASY	APR
ANN	–	–	225 $\pm$ 17	880 $\pm$ 0	733 $\pm$ 4	976 $\pm$ 1	751 $\pm$ 9	929 $\pm$ 19	100.00%
LIF	8	–	104.0	774.1	515.5	416.2	259.7	403.4	54.19%
TC-LIF	8	–	106.5	667.7	517.8	657.8	302.2	613.2	61.25%
SPIKING-WM	8	–	113.7	791.0	577.2	682.0	350.7	701.3	68.54%
IF	32	$\times$	57 $\pm$ 9	563 $\pm$ 99	240 $\pm$ 34	606 $\pm$ 39	295 $\pm$ 25	197 $\pm$ 59	40.77%
		$\checkmark$	167 $\pm$ 21	823 $\pm$ 8	411 $\pm$ 48	831 $\pm$ 11	752 $\pm$ 22	334 $\pm$ 58	74.19%
	64	$\times$	93 $\pm$ 12	843 $\pm$ 16	494 $\pm$ 42	896 $\pm$ 12	643 $\pm$ 26	331 $\pm$ 24	69.60%
		$\checkmark$	223 $\pm$ 24	867 $\pm$ 1	642 $\pm$ 20	940 $\pm$ 5	774 $\pm$ 21	617 $\pm$ 29	91.84%
SNM	8	$\times$	216 $\pm$ 19	853 $\pm$ 4	612 $\pm$ 30	913 $\pm$ 9	772 $\pm$ 24	551 $\pm$ 33	88.72%
		$\checkmark$	248 $\pm$ 25	853 $\pm$ 4	637 $\pm$ 23	933 $\pm$ 5	774 $\pm$ 18	787 $\pm$ 35	96.27%
	16	$\times$	217 $\pm$ 21	879 $\pm$ 0	714 $\pm$ 6	963 $\pm$ 6	758 $\pm$ 22	907 $\pm$ 17	98.50%
		$\checkmark$	237 $\pm$ 26	879 $\pm$ 0	728 $\pm$ 9	972 $\pm$ 1	781 $\pm$ 6	933 $\pm$ 19	101.41%
MT	2	$\times$	210 $\pm$ 17	878 $\pm$ 0	558 $\pm$ 43	952 $\pm$ 5	752 $\pm$ 29	947 $\pm$ 17	94.85%
		$\checkmark$	244 $\pm$ 28	878 $\pm$ 0	701 $\pm$ 7	954 $\pm$ 1	783 $\pm$ 8	947 $\pm$ 17	101.30%
DC	2	$\times$	215 $\pm$ 20	877 $\pm$ 0	620 $\pm$ 5	969 $\pm$ 1	749 $\pm$ 20	899 $\pm$ 17	95.97%
		$\checkmark$	248 $\pm$ 29	877 $\pm$ 0	660 $\pm$ 25	969 $\pm$ 1	768 $\pm$ 32	939 $\pm$ 20	100.43%

Table 4. Average inference-time energy consumption of ANNs and converted SNNs in DeepMind Control suite.

	ANN	IF (T=32)	MT (T=2)
FLOPs	4.53×10^7	–	–
SOPs	–	2.12×10^8	2.71×10^7
CONSUMPTIONS	566.84 μ J	16.35 μ J	2.09 μ J

the spiking world model (Spiking-WM) (Sun et al., 2025). CRPI consistently outperforms these directly trained SNN approaches, highlighting the advantage of leveraging well-trained ANN policies while effectively mitigating error accumulation and amplification during the conversion process.

6.5. Energy Efficiency

We further analyze the inference-time energy consumption of the converted SNN models. Following the widely adopted

estimation framework in (Merolla et al., 2014), we approximate energy expenditure by assigning 12.5 pJ per floating-point operation (FLOP) and 77 fJ per synaptic operation (SOP) (Qiao et al., 2015; Hu et al., 2021).

As shown in Table 4, ANN baselines incur substantially higher energy consumption per inference compared to their converted SNN counterparts. Despite requiring multiple simulation steps, SNNs achieve significant energy savings due to their sparse, event-driven computation. With advanced multi-threshold neurons, both the number of operations and the energy consumption are further reduced.

It is worth noting that the membrane potential initialization mechanism in CRPI introduces negligible energy overhead (less than 1%) compared to vanilla SNNs. More detailed results and discussions regarding energy efficiency can be found in Appendix B.4.

These results highlight the energy efficiency of SNNs and

further support the suitability of CRPI-converted SNNs for low-power and resource-constrained deployment scenarios.

7. Conclusion

This work presents a systematic study of ANN-to-SNN conversion in continuous control and identifies a fundamental limitation absent in classification and discrete control tasks: small approximation errors become temporally correlated through long-horizon interactions, inducing progressive state drift and severe performance degradation. To mitigate this effect, we propose Cross-Step Residual Potential Initialization, a gradient-free inference mechanism that suppresses cross-step error correlation. CRPI is compatible with diverse neuron models and conversion schemes, and consistently improves performance on MuJoCo and DMC benchmarks while preserving energy efficiency. Our findings position continuous control as a critical benchmark for evaluating ANN-to-SNN conversion, where even minor approximation errors can be greatly amplified and result in severe performance degradation. Future work will focus on developing adaptive tuning schemes for the correlation parameter α and extending the CRPI framework to broader sequential generation tasks vulnerable to error accumulation, such as language modeling and diffusion processes.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (62422601, U24B20140, 62506011), Beijing Municipal Science and Technology Program (Z251100008125052), and Qiyuan Innovative Talent Program.

Impact Statement

This work aims to advance ANN-to-SNN conversion for continuous control by analyzing the sources of conversion errors and proposing an inference-time method to mitigate temporally correlated error accumulation. By improving the stability and effectiveness of converted SNN policies in long-horizon decision-making tasks, this work may facilitate the deployment of energy-efficient spiking models in resource-constrained control systems such as robotics and embedded platforms. We **do not** anticipate significant ethical or societal risks beyond those commonly associated with control and reinforcement learning applications.

References

Bellec, G., Scherr, F., Subramoney, A., Hajek, E., Salaj, D., Legenstein, R., and Maass, W. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11(1):3625, 2020.

Brockman, G. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

Brunke, L., Greeff, M., Hall, A. W., Yuan, Z., Zhou, S., Panerati, J., and Schoellig, A. P. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5(1):411–444, 2022.

Bu, T., Ding, J., Yu, Z., and Huang, T. Optimized potential initialization for low-latency spiking neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, 2022a.

Bu, T., Fang, W., Ding, J., Dai, P., Yu, Z., and Huang, T. Optimal ANN-SNN conversion for high-accuracy and ultra-low-latency spiking neural networks. In *International Conference on Learning Representations*, 2022b.

Bu, T., Fang, W., Ding, J., Dai, P., Yu, Z., and Huang, T. Optimal ann-snn conversion for high-accuracy and ultra-low-latency spiking neural networks. *arXiv preprint arXiv:2303.04347*, 2023.

Bu, T., Li, M., and Yu, Z. Inference-scale complexity in ANN-SNN conversion for high-performance and low-power applications. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 24387–24397, 2025.

Cao, Y., Chen, Y., and Khosla, D. Spiking deep convolutional neural networks for energy-efficient object recognition. *International Journal of Computer Vision*, 2015.

Chen, D., Peng, P., Huang, T., and Tian, Y. Deep reinforcement learning with spiking q-learning. *arXiv preprint arXiv:2201.09754*, 2022.

Chen, D., Peng, P., Huang, T., and Tian, Y. Fully spiking actor network with intralayer connections for reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 36(2):2881–2893, 2024.

Davies, M., Srinivasa, N., Lin, T.-H., Chinya, G., Cao, Y., Choday, S. H., Dimou, G., Joshi, P., Imam, N., Jain, S., et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 2018.

DeBole, M. V., Taba, B., Amir, A., Akopyan, F., Andreopoulos, A., Risk, W. P., Kuszitz, J., Otero, C. O., Nayak, T. K., Appuswamy, R., et al. TrueNorth: Accelerating from zero to 64 million neurons in 10 years. *Computer*, 2019.

Deng, S. and Gu, S. Optimal conversion of conventional artificial neural networks to spiking neural networks. *arXiv preprint arXiv:2103.00476*, 2021.

- Ding, J., Dong, B., Heide, F., Ding, Y., Zhou, Y., Yin, B., and Yang, X. Biologically inspired dynamic thresholds for spiking neural networks. *Advances in Neural Information Processing Systems*, 35:6090–6103, 2022.
- Ding, J., Yu, Z., Liu, J. K., and Huang, T. Neuromorphic computing paradigms enhance robustness through spiking neural networks. *Nature Communications*, 16(1): 10175, 2025.
- Erez, T., Tassa, Y., and Todorov, E. Infinite-horizon model predictive control for periodic tasks with contacts. *Robotics: Science and Systems VII*, 2012.
- Feng, S., Cao, J., Ou, Z., Chen, G., Zhong, Y., Wang, Z., Yan, J., Chen, J., Wang, B., Zou, C., et al. Brainqn: Enhancing the robustness of deep reinforcement learning with spiking neural networks. *Advanced Intelligent Systems*, 6(9):2400075, 2024.
- Florian, R. V. Reinforcement learning through modulation of spike-timing-dependent synaptic plasticity. *Neural Computation*, 19(6):1468–1502, 2007.
- Frémaux, N. and Gerstner, W. Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules. *Frontiers in Neural Circuits*, 9:85, 2016.
- Frémaux, N., Sprekeler, H., and Gerstner, W. Reinforcement learning using a continuous time actor-critic framework with spiking neurons. *PLoS Computational Biology*, 9(4): e1003024, 2013.
- Fujimoto, S., Hoof, H., and Meger, D. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, pp. 1587–1596. PMLR, 2018.
- Gerstner, W., Kistler, W. M., Naud, R., and Paninski, L. *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press, 2014.
- Gerstner, W., Lehmann, M., Liakoni, V., Corneil, D., and Brea, J. Eligibility traces and plasticity on behavioral time scales: experimental support of neohebbian three-factor learning rules. *Frontiers in Neural Circuits*, 12:53, 2018.
- Gu, S., Holly, E., Lillicrap, T., and Levine, S. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *2017 IEEE International Conference on Robotics and Automation*, pp. 3389–3396. IEEE, 2017.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pp. 1861–1870. PMLR, 2018a.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018b.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- Han, B., Srinivasan, G., and Roy, K. Rmp-snn: Residual membrane potential neuron for enabling deeper high-accuracy and low-latency spiking neural network. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 13558–13567, 2020.
- Hao, Z., Bu, T., Ding, J., Huang, T., and Yu, Z. Reducing ann-snn conversion error through residual membrane potential. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pp. 11–21, 2023a.
- Hao, Z., Ding, J., Bu, T., Huang, T., and Yu, Z. Bridging the gap between anns and snns by calibrating offset spikes. *arXiv preprint arXiv:2302.10685*, 2023b.
- Hu, Y., Tang, H., and Pan, G. Spiking deep residual networks. *IEEE Transactions on Neural Networks and Learning Systems*, 34(8):5200–5205, 2021.
- Hu, Y., Zheng, Q., Jiang, X., and Pan, G. Fast-snn: Fast spiking neural network by converting quantized ann. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(12):14546–14562, 2023.
- Huang, Z., Shi, X., Hao, Z., Bu, T., Ding, J., Yu, Z., and Huang, T. Towards high-performance spiking transformers from ann to snn conversion. In *Proceedings of the 32nd ACM international conference on multimedia*, pp. 10688–10697, 2024.
- Huang, Z., Fang, W., Bu, T., Xue, P., Hao, Z., Liu, W., Tang, Y., Yu, Z., and Huang, T. Differential coding for training-free ann-to-snn conversion. *arXiv preprint arXiv:2503.00301*, 2025.
- Jayant, A. K. and Bhatnagar, S. Model-based safe deep reinforcement learning via a constrained proximal policy optimization algorithm. *Advances in Neural Information Processing Systems*, 35:24432–24445, 2022.
- Jiang, M., Rocktäschel, T., and Grefenstette, E. General intelligence requires rethinking exploration. *Royal Society Open Science*, 10(6):230539, 2023.

- Jiang, Y., Hu, K., Zhang, T., Gao, H., Liu, Y., Fang, Y., and Chen, F. Spatio-temporal approximation: A training-free snn conversion for transformers. In *The twelfth international conference on learning representations*, 2024.
- Kim, J., Kim, H., Huh, S., Lee, J., and Choi, K. Deep neural networks with weighted spikes. *Neurocomputing*, 311: 373–386, 2018.
- Kober, J., Bagnell, J. A., and Peters, J. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.
- Kumar, A., Zhang, L., Bilal, H., Wang, S., Shaikh, A. M., Bo, L., Rohra, A., and Khalid, A. Dsqn: Robust path planning of mobile robot based on deep spiking q-network. *Neurocomputing*, 634:129916, 2025.
- Li, C., Ma, L., and Furber, S. Quantization framework for fast spiking neural networks. *Frontiers in Neuroscience*, 16:918793, 2022.
- Li, J., Sun, Y., Ma, Z., He, H., Zhang, X., Chen, X., Li, J., and Song, S. Signal-adaptive trust regions for gradient-free optimization of recurrent spiking neural networks. *arXiv preprint arXiv:2601.21572*, 2026.
- Li, Y. and Zeng, Y. Efficient and accurate conversion of spiking neural network with burst spikes. *arXiv preprint arXiv:2204.13271*, 2022.
- Li, Y., Deng, S., Dong, X., Gong, R., and Gu, S. A free lunch from ann: Towards efficient, accurate spiking neural networks calibration. In *International conference on machine learning*, pp. 6316–6325. PMLR, 2021.
- Lillicrap, T. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Liu, G., Deng, W., Xie, X., Huang, L., and Tang, H. Human-level control through directly trained deep spiking q-networks. *IEEE Transactions on Cybernetics*, 53(11): 7187–7198, 2022.
- Maass, W. Networks of spiking neurons: the third generation of neural network models. *Neural Networks*, 10(9): 1659–1671, 1997.
- Merolla, P. A., Arthur, J. V., Alvarez-Icaza, R., Cassidy, A. S., Sawada, J., Akopyan, F., Jackson, B. L., Imam, N., Guo, C., Nakamura, Y., et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.
- Mnih, V. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.
- Oh, H. and Lee, Y. Sign gradient descent-based neuronal dynamics: Ann-to-snn conversion beyond relu network. *arXiv preprint arXiv:2407.01645*, 2024.
- Padalkar, A., Quere, G., Raffin, A., Silv rio, J., and Stulp, F. Guiding real-world reinforcement learning for in-contact manipulation tasks with shared control templates. *Autonomous Robots*, 48(4):12, 2024.
- Park, S., Kim, S., Choe, H., and Yoon, S. Fast and efficient information transmission with burst spikes in deep spiking neural networks. In *Proceedings of the 56th Annual Design Automation Conference 2019*, pp. 1–6, 2019.
- Patel, D., Hazan, H., Saunders, D. J., Siegelmann, H. T., and Kozma, R. Improved robustness of reinforcement learning policies upon conversion to spiking neuronal network platforms applied to atari breakout game. *Neural Networks*, 120:108–115, 2019.
- Qiao, N., Mostafa, H., Corradi, F., Osswald, M., Stefanini, F., Sumislawska, D., and Indiveri, G. A reconfigurable on-line learning spiking neuromorphic processor comprising 256 neurons and 128K synapses. *Frontiers in Neuroscience*, 9:141, 2015.
- Qin, L., Yan, R., and Tang, H. A low latency adaptive coding spiking framework for deep reinforcement learning. *arXiv preprint arXiv:2211.11760*, 2022.
- Qin, L., Wang, Z., Jiang, R., Yan, R., and Tang, H. Grsn: Gated recurrent spiking neurons for pomdps and marl. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 1483–1491, 2025.
- Rueckauer, B. and Liu, S.-C. Conversion of analog to spiking neural networks using sparse temporal coding. In *2018 IEEE international symposium on circuits and systems (ISCAS)*, pp. 1–5. IEEE, 2018.
- Rueckauer, B., Lungu, I.-A., Hu, Y., Pfeiffer, M., and Liu, S.-C. Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in neuroscience*, 11:682, 2017.
- Schulman, J. Trust region policy optimization. *arXiv preprint arXiv:1502.05477*, 2015.
- Song, R., Nie, D., Teng, S., Ding, B., Zhang, X., Li, D., Zhang, C., Li, Y., Wu, H., and Chen, L. SpikeVLA: Vision-language-action models with spiking neural networks. In *Forty-third International Conference on Machine Learning*, 2026.

- Stanojevic, A., Woźniak, S., Bellec, G., Cherubini, G., Pantazi, A., and Gerstner, W. An exact mapping from relu networks to spiking neural networks. *Neural Networks*, 168:74–88, 2023.
- Sun, Y., Zeng, Y., and Li, Y. Solving the spike feature information vanishing problem in spiking deep q network with potential based normalization. *Frontiers in Neuroscience*, 16:953368, 2022.
- Sun, Y., Zhao, F., Lyu, M., and Zeng, Y. Spiking world model with multicompartment neurons for model-based reinforcement learning. *Proceedings of the National Academy of Sciences*, 122(50):e2513319122, 2025.
- Tan, W., Patel, D., and Kozma, R. Strategy and benchmark for converting deep q-networks to event-driven spiking neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 9816–9824, 2021.
- Tang, C., Abbatematteo, B., Hu, J., Chandra, R., Martín-Martín, R., and Stone, P. Deep reinforcement learning for robotics: A survey of real-world successes. *Annual Review of Control, Robotics, and Autonomous Systems*, 8(1):153–188, 2025.
- Tang, G., Kumar, N., and Michmizos, K. P. Reinforcement co-learning of deep and spiking neural networks for energy-efficient mapless navigation with neuromorphic hardware. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 6090–6097. IEEE, 2020.
- Tang, G., Kumar, N., Yoo, R., and Michmizos, K. Deep reinforcement learning with population-coded spiking neural network for continuous control. In *Conference on Robot Learning*, pp. 2016–2029. PMLR, 2021.
- Todorov, E. Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in mujoco. In *2014 IEEE International Conference on Robotics and Automation*, pp. 6054–6061. IEEE, 2014.
- Todorov, E., Erez, T., and Tassa, Y. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012.
- Towers, M., Kwiatkowski, A., Terry, J. K., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., Arjun, K., et al. Gymnasium: A standard interface for reinforcement learning environments. *CoRR*, 2024.
- Tunyasuvunakool, S., Muldal, A., Doron, Y., Liu, S., Bohez, S., Merel, J., Erez, T., Lillicrap, T., Heess, N., and Tassa, Y. dm.control: Software and tasks for continuous control. *Software Impacts*, 6:100022, 2020.
- Wang, Y., Zhang, M., Chen, Y., and Qu, H. Signed neuron with memory: Towards simple, accurate and high-efficient ann-snn conversion. In *IJCAI*, pp. 2501–2508, 2022a.
- Wang, Z., Gu, X., Goh, R. S. M., Zhou, J. T., and Luo, T. Efficient spiking neural networks with radix encoding. *IEEE Transactions on Neural Networks and Learning Systems*, 35(3):3689–3701, 2022b.
- Wang, Z., Fang, Y., Cao, J., Zhang, Q., Wang, Z., and Xu, R. Masked spiking transformer. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 1761–1771, 2023.
- Wang, Z., Fang, Y., Cao, J., Ren, H., and Xu, R. Adaptive calibration: A unified conversion framework of spiking neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 1583–1591, 2025.
- Wawrzyński, P. A cat-like robot real-time learning to run. In *Adaptive and Natural Computing Algorithms: 9th International Conference, ICANNGA 2009, Kuopio, Finland, April 23-25, 2009, Revised Selected Papers 9*, pp. 380–390. Springer, 2009.
- Wu, Y., Deng, L., Li, G., Zhu, J., and Shi, L. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, 12: 331, 2018.
- Xu, Z., Bu, T., Hao, Z., Ding, J., and Yu, Z. Proxy target: Bridging the gap between discrete spiking neural networks and continuous control. *Advances in Neural Information Processing Systems*, 38:159158–159184, 2026a.
- Xu, Z., Shi, X., Dong, Y., Huang, Z., and Yu, Z. CaRe-BN: Precise moving statistics for stabilizing spiking neural networks in reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026b.
- Yang, Z., Guo, S., Fang, Y., Yu, Z., and Liu, J. K. Spiking variational policy gradient for brain inspired reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- Yarats, D., Fergus, R., Lazaric, A., and Pinto, L. Mastering visual continuous control: Improved data-augmented reinforcement learning. *arXiv preprint arXiv:2107.09645*, 2021a.
- Yarats, D., Kostrikov, I., and Fergus, R. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *International Conference on Learning Representations*, 2021b.

You, K., Xu, Z., Nie, C., Deng, Z., Guo, Q., Wang, X., and He, Z. Spikezip-tf: Conversion is all you need for transformer-based SNN. *arXiv preprint arXiv:2406.03470*, 2024.

Zhang, L., Zhou, S., Zhi, T., Du, Z., and Chen, Y. Tdsnn: From deep neural networks to deep spike neural networks with temporal-coding. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pp. 1319–1326, 2019.

Zhang, S., Yang, Q., Ma, C., Wu, J., Li, H., and Tan, K. C. Tc-lif: A two-compartment spiking neuron model for long-term sequential modelling. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 16838–16847, 2024.

A. Additional Experiments Details

A.1. Reinforcement Learning Environments

We evaluate the proposed method on a diverse set of continuous control benchmarks covering both vector-based and vision-based observations. Specifically, we consider standard MuJoCo (Todorov et al., 2012; Todorov, 2014) tasks from OpenAI Gymnasium (Brockman, 2016; Towers et al., 2024) and visual control tasks from the DeepMind Control Suite (DMC) (Tunyasuvunakool et al., 2020). These environments are widely used for benchmarking reinforcement learning algorithms.

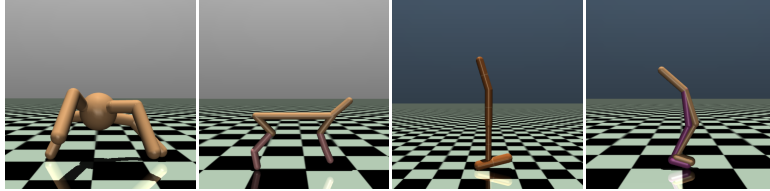


Figure 6. Representative MuJoCo continuous control tasks used in our experiments. From left to right: Ant-v4, HalfCheetah-v4, Hopper-v4, and Walker2d-v4.

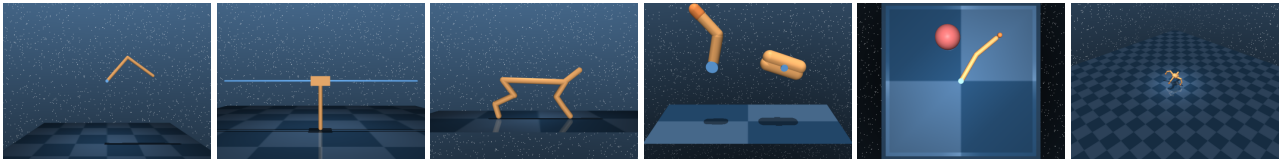


Figure 7. Representative DeepMind Control Suite tasks used in our experiments. From left to right: acrobot_swingup, cartpole_swingup, cheetah_run, finger_spin, reacher_easy, and quadruped_walk.

As shown in Figure 6, the MuJoCo environments includes Ant (Schulman, 2015), HalfCheetah (Wawrzyński, 2009), Hopper (Erez et al., 2012), and Walker2d. Besides, the DMC Suite includes cartpole_swingup, finger_spin, reacher_easy, cheetah_run, acrobot_swingup, and quadruped_walk, demonstrated in Figure 7.

Table 5. State and action space dimensions for MuJoCo environments.

ENVIRONMENT	STATE DIMENSION	ACTION DIMENSION
ANT-V4	27	8
HALFCHEETAH-V4	17	6
HOPPER-V4	11	3
WALKER2D-V4	17	6

Tables 5 and 6 summarize the dimensionalities of state and action spaces for all evaluated environments. MuJoCo tasks use low-dimensional vector states, while DMC tasks rely on pixel observations; in all cases, the action space is continuous.

All environments are used with their default parameters provided by the respective simulators. Rewards are not rescaled or normalized during either training or evaluation. For evaluation, each episode is capped at a maximum horizon of 1000 environment interactions, unless terminated earlier by environment-specific conditions. These settings are kept consistent across ANN baselines and converted SNN policies to ensure fair comparison.

A.2. Reinforcement Learning Algorithms

For vector-based continuous control tasks in MuJoCo, we employ three widely used off-policy actor-critic algorithms: Deep Deterministic Policy Gradient (DDPG) (Lillicrap, 2015), Twin Delayed DDPG (TD3) (Fujimoto et al., 2018), and Soft Actor-Critic (SAC) (Haarnoja et al., 2018a;b). All three methods learn a deterministic (DDPG, TD3) or stochastic (SAC) policy together with one or more Q-function critics. The actor networks in all methods are implemented as multilayer

Table 6. Observation and action space specifications for DeepMind Control Suite environments.

DOMAIN NAME	TASK NAME	OBSERVATION	ACTION DIMENSION
ACROBOT	SWINGUP	$84 \times 84 \times 3$	1
CARTPOLE	SWINGUP	$84 \times 84 \times 3$	1
FINGER	SPIN	$84 \times 84 \times 3$	2
REACHER	EASY	$84 \times 84 \times 3$	2
CHEETATH	RUN	$84 \times 84 \times 3$	6
QUADRUPED	WALK	$84 \times 84 \times 3$	12

perceptrons (MLPs) with two hidden layers and ReLU activations. DDPG has 400 hidden units in layer 1 and 300 hidden units in layer 2. TD3 and SAC have 256 units in both hidden layers.

For image-based continuous control tasks in the DeepMind Control Suite, we adopt Data-Regularized Q-v2 (DrQ-v2) (Yarats et al., 2021b;a), a sample-efficient off-policy algorithm designed for high-dimensional visual observations. DrQ-v2 employs a convolutional encoder followed by an actor-critic architecture. Specifically, the visual encoder consists of four convolutional layers with 3×3 kernels and 32 channels, followed by a fully connected layer that produces a compact latent representation of 50 dimensions. Both the actor and critic networks operate on this latent feature and are implemented as two-layer MLPs with 1024 hidden units.

All RL agents are trained using standard hyperparameters and default environment settings. After training, the actor networks are converted to SNNs using different ANN-to-SNN conversion methods. During evaluation, only the converted SNN policies interact with the environment, and no further learning or fine-tuning is performed.

A.3. ANN-to-SNN Conversion Approaches

Our experiments evaluate three baseline methods: Signed Neuron with Memory (SNM) (Wang et al., 2022a), Multi Threshold (MT) Neuron (Huang et al., 2024), and Differential coding (DC) based neuron (Huang et al., 2025).

A.3.1. SNM NEURON DYNAMICS

SNM neuron can be regarded as an IF neuron with negative threshold and more strict spike emission condition on negative threshold in SNNs, let $\mathbf{m}^l(t)$ and $\mathbf{v}^l(t)$ denote the membrane potential of neurons in the l -th layer before and after firing spikes at time-step t , the neural dynamic can be formulated as follows:

$$\mathbf{m}^l(t) = \mathbf{v}^l(t-1) + \mathbf{W}^l \mathbf{x}^{l-1}(t), \quad (23)$$

$$\mathbf{s}^l(t) = H(\mathbf{m}^l(t) - \theta^l) - H(-\mathbf{m}^l(t) + \theta^l) \cdot H(\mathbf{c}^l(t) + \theta^l), \quad (24)$$

$$\mathbf{x}^l(t) = \theta^l \mathbf{s}^l(t), \quad (25)$$

$$\mathbf{v}^l(t) = \mathbf{m}^l(t) - \mathbf{x}^l(t). \quad (26)$$

$$\mathbf{c}^l[t] = \mathbf{c}^l[t-1] + \mathbf{x}^l[t], \quad (27)$$

where H is the Heaviside step function and θ^l is the neuron threshold in layer l . $\mathbf{s}^l(t)$ is the output spike of layer l . $\mathbf{x}^l(t)$ is the postsynaptic potential and theoretical output of layer l . $\mathbf{c}^l[t-1]$ represents an auxiliary cumulative variable used to support the ReLU-like behavior.

A.3.2. MT NEURON DYNAMICS

The MT neuron is characterized by several parameters, including the base threshold θ , and a total of $2n$ thresholds, with n positive and n negative thresholds. The threshold values of the MT neuron are indexed by i , where λ_i^l represents the i -th threshold value in the layer l :

$$\begin{aligned} \lambda_1^l &= \theta^l, \lambda_2^l = \frac{\theta^l}{2}, \dots, \lambda_n^l = \frac{\theta^l}{2^{n-1}}, \\ \lambda_{n+1}^l &= -\theta^l, \lambda_{n+2}^l = -\frac{\theta^l}{2}, \dots, \lambda_{2n}^l = -\frac{\theta^l}{2^{n-1}}. \end{aligned} \quad (28)$$

Let variables $\mathbf{I}^l[t]$, $\mathbf{W}^l$, $s_i^l[t]$, $\mathbf{x}^l[t]$, $\mathbf{m}^l[t]$, and $\mathbf{v}^l[t]$ represent the input current, weight, the output spike of the i -th threshold, the total output signal, and the membrane potential before and after spikes in the l -th layer at the time-step t . It defines $\frac{4}{3}\mathbf{m}^l[t] = (-1)^{S_2} 2^{E_2} (1 + M_2)$ with 1 sign bit (S), 8 exponent bits (E), and 23 mantissa bits (M). Since the median of $\frac{1}{2^{k-1}}$ and $\frac{1}{2^k}$ is $\frac{3}{4} \frac{1}{2^{k-1}}$, we can easily select the correct threshold index i using E and S of $\frac{4}{3}\mathbf{m}^l[t]$. The dynamics of the MT neurons are described by the following equations:

$$\mathbf{m}^l[t] = \mathbf{v}^l[t-1] + \mathbf{I}^l[t] = \mathbf{v}^l[t-1] + \mathbf{x}^{l-1}[t], \quad (29)$$

$$s_i^l[t] = \text{MTH-R}_{\theta,n}(\mathbf{m}^l[t], i) \quad (30)$$

$$\mathbf{x}^l[t] = \sum_i s_i^l[t] \mathbf{W}^l \lambda_i^l, \quad (31)$$

$$\mathbf{v}^l[t] = \mathbf{m}^l[t] - \mathbf{x}^l[t], \quad (32)$$

$$\text{MTH-R}_{\theta,n}(\mathbf{m}^l[t], i) = \begin{cases} 1, & \text{if } \begin{cases} i < n, & S = 0 \text{ and } i - 1 = -E, \\ i \geq n, & S = 1 \text{ and } \\ & i - n - 1 = \max(-E, -E_2) \end{cases} \\ 0, & \text{otherwise.} \end{cases} \quad (33)$$

$$(34)$$

$$\mathbf{c}^l[t] = \mathbf{c}^l[t-1] + \mathbf{x}^l[t], \quad (35)$$

where $\mathbf{c}^l[t-1] = (-1)^{S_2} 2^{E_2} (1 + M_2)$ represents an auxiliary cumulative variable used to support the ReLU-like behavior.

A.3.3. DC BASED NEURON DYNAMICS

In rate coding, the output of the previous layer, $\mathbf{x}^{l-1}[t]$, is directly used as the input current for the current layer $\mathbf{I}^l[t] = \mathbf{x}^{l-1}[t]$. In differential coding, the input current $\mathbf{I}^l[t]$ can be adjusted as shown in Equation (36), which converts any spiking neuron into a differential spiking neuron:

$$\mathbf{I}^l[t] = \mathbf{m}_r^l[t] + \mathbf{x}^{l-1}[t], \quad (36)$$

$$\mathbf{m}_r^l[t+1] = \mathbf{m}_r^l[t] + \frac{\mathbf{x}^{l-1}[t]}{t} - \frac{\mathbf{x}^l[t]}{t}, \quad (37)$$

where $\mathbf{m}_r^l[0]$ is $\mathbf{b}^{l-1}$ if the previous layer has bias else 0. This work employs differential coding methods based on MT neurons.

For linear layers, including fully connected and convolutional layers that can be represented by Equation (38),

$$\mathbf{x}^l = \mathbf{W}^l \mathbf{x}^{l-1} + \mathbf{b}^l, \quad (38)$$

where $\mathbf{W}^l$ and $\mathbf{b}^l$ is the weight and bias of layer l . Under differential coding in SNNs, this is equivalent to eliminating the bias term $\mathbf{b}^l$ and initializing the membrane potential of the subsequent layer with the bias value as Equation (39):

$$\mathbf{x}^l = \mathbf{W}^l \mathbf{x}^{l-1}. \quad (39)$$

B. Additional Experiments Results

B.1. Additional Results on Reward Decomposition

This section provides additional empirical results for the reward decomposition analysis introduced in Section 4.1. The expected return is decomposed into four counterfactual settings: (i) the original ANN policy and state trajectory R^{ANN} , (ii) the fully converted SNN policy and state trajectory R^{SNN} , (iii) ANN policy evaluated on SNN-induced state trajectories $R^{\text{ANN|SNN}}$, and (iv) SNN policy evaluated on ANN-induced state trajectories $R^{\text{SNN|ANN}}$.

Table 7 reports detailed results for Integrate-and-Fire (IF) neurons converted from TD3 policies on MuJoCo environments under different SNN simulation time steps. Across all environments and time horizons, we observe a consistent pattern that replacing the policy alone while keeping ANN state trajectories results in only marginal performance degradation, whereas

Table 7. Reward decomposition results for ANN-to-SNN conversion using IF neurons. ANNs are trained with TD3 in MuJoCo environments for 3 million interactions.

ENVIRONMENT	T	R^{ANN}	$R^{\text{SNN ANN}}$	$R^{\text{ANN SNN}}$	R^{SNN}
ANT-v4	8		6216.71	4051.77	3988.25
	16	6505.26	6393.15	6112.45	6008.63
	32		6475.02	6263.13	6294.63
HALFCHEETAH-v4	8		13164.41	4146.40	4104.06
	16	13193.35	13179.90	6477.50	6445.50
	32		13190.03	9720.97	9711.38
HOPPER-v4	8		3605.06	3519.23	3532.34
	16	3594.20	3602.29	3560.10	3572.05
	32		3599.08	3575.31	3580.73
WALKER2D-v4	8		4620.69	3118.59	3122.68
	16	4582.30	4610.12	3471.64	3475.12
	32		4598.57	4058.01	4065.86

replacing the state trajectory leads to substantial return loss, even when the ANN policy is used. This further confirms that state distribution shift is the dominant factor driving performance degradation in ANN-to-SNN conversion for continuous control.

These results align with the main findings in the paper and demonstrate that the state-dominated performance gap persists across different environments and SNN time resolutions.

B.2. Distribution of the Final Membrane Potential

In Section 5.1, we derive the CRPI mechanism under the assumption that the final membrane potential is approximately uniformly distributed in $(0, \theta^l)$ (Bu et al., 2022a) and weakly dependent on its initialization. To empirically validate this assumption, we analyze the distribution of final membrane potentials for a converted TD3 agent (IF neuron, $T=64$) in the Hopper-v4 environment.

Table 8. Distribution of final membrane potentials under different initializations.

INITIALIZATION	$(-\infty, 0]$	$(0, 0.1\theta]$	$(0.1\theta, 0.2\theta]$	$(0.2\theta, 0.3\theta]$	$(0.3\theta, 0.4\theta]$	$(0.4\theta, 0.5\theta]$	$(0.5\theta, 0.6\theta]$	$(0.6\theta, 0.7\theta]$	$(0.7\theta, 0.8\theta]$	$(0.8\theta, 0.9\theta]$	$(0.9\theta, \theta]$	$(\theta, +\infty)$
0	85.3%	1.4%	1.5%	1.5%	1.5%	1.5%	1.5%	1.4%	1.5%	1.4%	1.4%	0.2%
0.25θ	85.2%	1.5%	1.5%	1.5%	1.5%	1.5%	1.4%	1.4%	1.5%	1.4%	1.4%	0.2%
0.5θ	85.2%	1.5%	1.5%	1.5%	1.5%	1.5%	1.4%	1.4%	1.4%	1.4%	1.4%	0.2%
0.75θ	85.1%	1.5%	1.5%	1.5%	1.5%	1.5%	1.5%	1.5%	1.4%	1.4%	1.4%	0.2%
θ	85.1%	1.5%	1.5%	1.5%	1.5%	1.5%	1.5%	1.5%	1.5%	1.4%	1.4%	0.2%

Table 8 shows that most neurons have negative potentials, where they are mostly inactive (and clipped in CRPI), and are thus irrelevant to the mechanism. For active neurons, membrane potentials are approximately uniformly distributed over $(0, \theta]$, with only a small fraction exceeding θ (and are also mostly clipped). Furthermore, this distribution remains nearly unchanged across initializations, indicating weak dependence on initialization.

B.3. Detailed MuJoCo Results across RL Algorithms

This section provides detailed per-environment results for ANN-to-SNN conversion on MuJoCo benchmarks, complementing the aggregated results reported in the main paper. Due to space constraints, the main text only presents averaged performance metrics across environments. Here, we report the full breakdown across individual tasks.

Note that DDPG fails to converge reliably on the Ant environment, which is consistent with prior observations in the literature (Xu et al., 2026a). As a result, results for DDPG are reported only on the remaining three MuJoCo tasks. TD3 and SAC results include all four environments.

Across all settings, CRPI consistently improves conversion performance relative to standard initialization, with gains observed across environments, time steps, and neuron types. These detailed results further support the robustness and generality of the proposed method.

Table 9. Detailed ANN-to-SNN conversion results on MuJoCo environments using DDPG, where $\pm$ captures half a standard deviation.

NEURON	TIME	CRPI	HALFCHEETAH	HOPPER	WALKER	APR
ANN	–	–	9126 $\pm$ 129	2703 $\pm$ 121	1982 $\pm$ 201	100.00%
IF	$T = 8$	$\times$	4486 $\pm$ 337	1097 $\pm$ 54	2856 $\pm$ 214	77.96%
		$\checkmark$	4486 $\pm$ 337	1410 $\pm$ 87	3212 $\pm$ 299	87.78%
	$T = 16$	$\times$	5744 $\pm$ 319	1689 $\pm$ 86	2424 $\pm$ 287	82.57%
		$\checkmark$	5799 $\pm$ 230	2021 $\pm$ 63	2931 $\pm$ 159	95.41%
	$T = 32$	$\times$	6591 $\pm$ 419	2311 $\pm$ 120	2477 $\pm$ 278	94.24%
		$\checkmark$	7321 $\pm$ 543	2815 $\pm$ 116	2593 $\pm$ 66	105.07%
SNM	$T = 8$	$\times$	5600 $\pm$ 434	1529 $\pm$ 75	1568 $\pm$ 263	65.69%
		$\checkmark$	6725 $\pm$ 650	1620 $\pm$ 63	1913 $\pm$ 259	76.72%
	$T = 16$	$\times$	8086 $\pm$ 490	2092 $\pm$ 136	2367 $\pm$ 178	95.14%
		$\checkmark$	8500 $\pm$ 435	2490 $\pm$ 154	2397 $\pm$ 329	102.06%
MT	$T = 2$	$\times$	7100 $\pm$ 605	1737 $\pm$ 38	1272 $\pm$ 164	68.74%
		$\checkmark$	7615 $\pm$ 556	1874 $\pm$ 107	2129 $\pm$ 336	86.74%
	$T = 4$	$\times$	8445 $\pm$ 350	2210 $\pm$ 113	1640 $\pm$ 140	85.68%
		$\checkmark$	9040 $\pm$ 356	2310 $\pm$ 231	2420 $\pm$ 219	102.22%
DC	$T = 2$	$\times$	6926 $\pm$ 493	1681 $\pm$ 79	1682 $\pm$ 262	74.32%
		$\checkmark$	7485 $\pm$ 518	1817 $\pm$ 104	2035 $\pm$ 142	83.97%
	$T = 4$	$\times$	8644 $\pm$ 222	2428 $\pm$ 214	2387 $\pm$ 68	101.65%
		$\checkmark$	8818 $\pm$ 405	2605 $\pm$ 191	2407 $\pm$ 189	104.81%

B.4. computational overhead of CRPI

During deployment, the parameter α in CRPI can be selected from the set 0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1, where each value can be expressed as a sum of powers of 1/2. The multiplication between α and the residual membrane potential can be efficiently implemented using at most three bitwise shifts and three floating-point accumulation operations (ACs). Other operations in CRPI (e.g., addition and clipping) do not involve multiplication. This design effectively eliminates multiplication operations during deployment, leading to minimal overhead.

Table 12 reports the average number of ACs introduced by CRPI compared to standard forward propagation. Across different neuron models, CRPI contributes less than 1% additional ACs, confirming its negligible overhead.

Error Amplification Limits ANN-to-SNN Conversion in Continuous Control

Table 10. Detailed ANN-to-SNN conversion results on MuJoCo environments using TD3, where $\pm$ captures half a standard deviation.

NEURON	T	CRPI	ANT	HALFCHEETAH	HOPPER	WALKER	APR
ANN	–	–	6505 $\pm$ 127	13193 $\pm$ 12	3594 $\pm$ 1	4582 $\pm$ 4	100.00%
IF	8	$\times$	3988 $\pm$ 526	4104 $\pm$ 522	3532 $\pm$ 3	3123 $\pm$ 275	64.71%
		$\checkmark$	4261 $\pm$ 236	4625 $\pm$ 400	3560 $\pm$ 2	4098 $\pm$ 130	72.26%
	16	$\times$	6009 $\pm$ 252	6445 $\pm$ 592	3572 $\pm$ 1	3475 $\pm$ 324	79.11%
		$\checkmark$	6009 $\pm$ 252	7261 $\pm$ 331	3576 $\pm$ 1	4285 $\pm$ 86	85.10%
SNM	32	$\times$	6295 $\pm$ 201	9711 $\pm$ 273	3581 $\pm$ 1	4066 $\pm$ 210	89.68%
		$\checkmark$	6666 $\pm$ 48	9938 $\pm$ 493	3586 $\pm$ 1	4422 $\pm$ 80	93.52%
	8	$\times$	5025 $\pm$ 358	10014 $\pm$ 600	3358 $\pm$ 105	3909 $\pm$ 161	82.97%
		$\checkmark$	5430 $\pm$ 456	10014 $\pm$ 600	3583 $\pm$ 4	4486 $\pm$ 70	89.24%
MT	16	$\times$	6138 $\pm$ 242	12168 $\pm$ 323	3592 $\pm$ 1	4594 $\pm$ 3	96.70%
		$\checkmark$	6423 $\pm$ 124	12365 $\pm$ 259	3594 $\pm$ 1	4595 $\pm$ 3	98.18%
	2	$\times$	2637 $\pm$ 325	10120 $\pm$ 178	3364 $\pm$ 92	4323 $\pm$ 105	76.29%
		$\checkmark$	2961 $\pm$ 254	10967 $\pm$ 237	3396 $\pm$ 98	4473 $\pm$ 78	80.18%
DC	4	$\times$	6359 $\pm$ 160	12720 $\pm$ 181	3592 $\pm$ 1	4473 $\pm$ 86	97.93%
		$\checkmark$	6359 $\pm$ 160	12720 $\pm$ 181	3592 $\pm$ 1	4501 $\pm$ 87	98.09%
	2	$\times$	6014 $\pm$ 145	11061 $\pm$ 130	3329 $\pm$ 98	4062 $\pm$ 220	89.39%
		$\checkmark$	6091 $\pm$ 138	12043 $\pm$ 166	3514 $\pm$ 36	4212 $\pm$ 153	93.65%
DC	4	$\times$	6408 $\pm$ 287	13033 $\pm$ 30	3593 $\pm$ 1	4579 $\pm$ 4	99.30%
		$\checkmark$	6538 $\pm$ 199	13033 $\pm$ 30	3593 $\pm$ 1	4579 $\pm$ 4	99.80%

Table 11. Detailed ANN-to-SNN conversion results on MuJoCo environments using SAC, where $\pm$ captures half a standard deviation.

NEURON	T	CRPI	ANT	HALFCHEETAH	HOPPER	WALKER	APR
ANN	–	–	6829 $\pm$ 140	14967 $\pm$ 22	3385 $\pm$ 89	5030 $\pm$ 70	100.00%
IF	8	$\times$	3258 $\pm$ 237	7960 $\pm$ 478	2966 $\pm$ 116	4651 $\pm$ 150	70.25%
		$\checkmark$	4127 $\pm$ 168	8043 $\pm$ 318	3017 $\pm$ 111	4941 $\pm$ 87	75.38%
	16	$\times$	5123 $\pm$ 284	11406 $\pm$ 213	3555 $\pm$ 7	4978 $\pm$ 90	88.80%
		$\checkmark$	5944 $\pm$ 164	11617 $\pm$ 239	3555 $\pm$ 7	5165 $\pm$ 21	93.09%
SNM	32	$\times$	6715 $\pm$ 98	13462 $\pm$ 110	3535 $\pm$ 30	4990 $\pm$ 62	97.98%
		$\checkmark$	6755 $\pm$ 98	13570 $\pm$ 88	3612 $\pm$ 3	5156 $\pm$ 15	99.70%
	8	$\times$	5709 $\pm$ 374	6584 $\pm$ 538	3155 $\pm$ 54	4901 $\pm$ 81	79.56%
		$\checkmark$	6539 $\pm$ 37	7662 $\pm$ 442	3493 $\pm$ 65	5101 $\pm$ 34	87.89%
MT	16	$\times$	6832 $\pm$ 141	10727 $\pm$ 494	3233 $\pm$ 192	5096 $\pm$ 36	92.13%
		$\checkmark$	6966 $\pm$ 9	12066 $\pm$ 242	3660 $\pm$ 30	5111 $\pm$ 11	98.09%
	2	$\times$	6579 $\pm$ 108	7908 $\pm$ 766	3037 $\pm$ 153	4889 $\pm$ 147	84.02%
		$\checkmark$	6628 $\pm$ 90	9073 $\pm$ 681	3476 $\pm$ 98	5022 $\pm$ 25	90.06%
DC	4	$\times$	6823 $\pm$ 122	11850 $\pm$ 308	3490 $\pm$ 114	4979 $\pm$ 80	95.29%
		$\checkmark$	7003 $\pm$ 11	11917 $\pm$ 508	3588 $\pm$ 103	5110 $\pm$ 21	97.44%
	2	$\times$	5729 $\pm$ 332	10123 $\pm$ 359	3204 $\pm$ 193	5037 $\pm$ 35	86.58%
		$\checkmark$	6375 $\pm$ 143	10955 $\pm$ 343	3518 $\pm$ 94	5091 $\pm$ 20	92.93%
DC	4	$\times$	6668 $\pm$ 195	13780 $\pm$ 302	3315 $\pm$ 120	5016 $\pm$ 82	96.85%
		$\checkmark$	7020 $\pm$ 12	14035 $\pm$ 69	3672 $\pm$ 22	5122 $\pm$ 18	101.72%

Table 12. Average computational overhead of CRPI on the DeepMind Control Suite.

SPIKING NEURON	T	ACS IN CRPI	ACS IN FORWARD PROPAGATION	CRPI OVERHEAD
IF	32	1.76×10^5	2.12×10^8	0.08%
MT	2	1.88×10^5	2.71×10^7	0.69%